Algorithms and Data Structures 2.

–

Breadth First Search – BFS

Harrach Nóra - harrachnora@inf.elte.hu

Practice 3

Table of Contents

Exercise

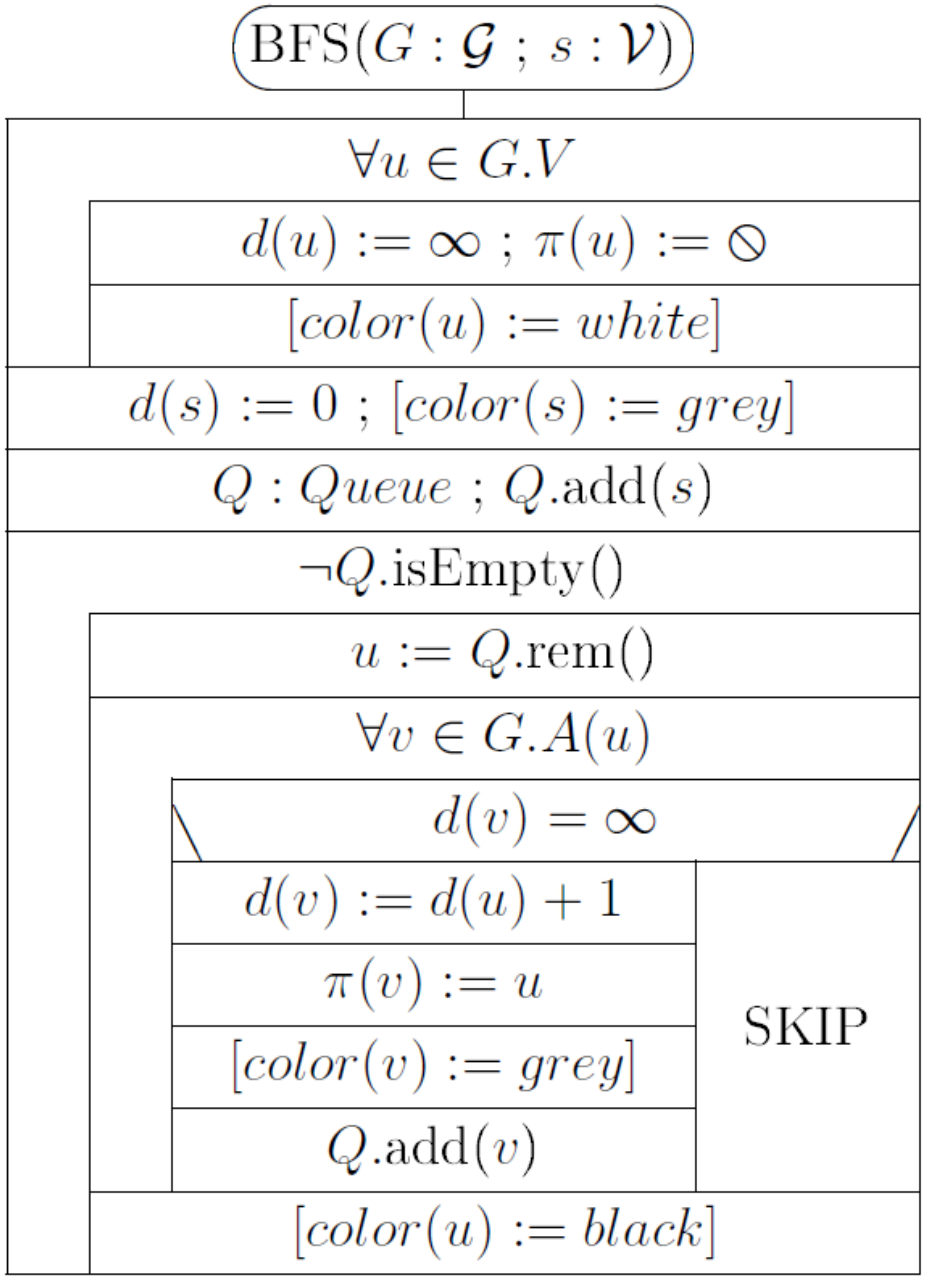
Design an algorithm which calculates the in-degrees and out-degrees of all the vertices of a graph given in (a) adjacency matrix representation or (b) adjacency list representation. Thes values should be given in two arrays $In/1$ and $Out/1$.

Table of Contents

Breadth-first search

- Breadth-first search (or BFS) will be considered on both undirected or directed graphs.
- We will fix a source vertex $s \in G.V$.
- We will find the shortest path to each other vertex available from s.
- If there are more than one shortest paths, then we only compute one of them.

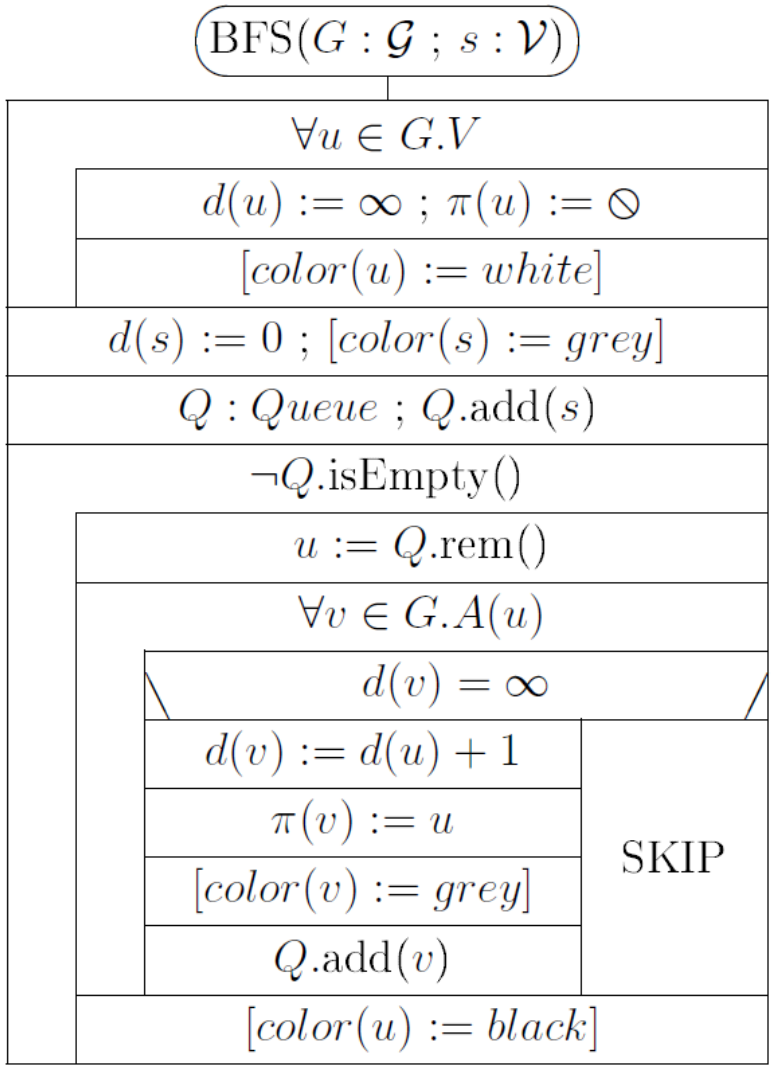
Breadth-first search



Breadth-first search

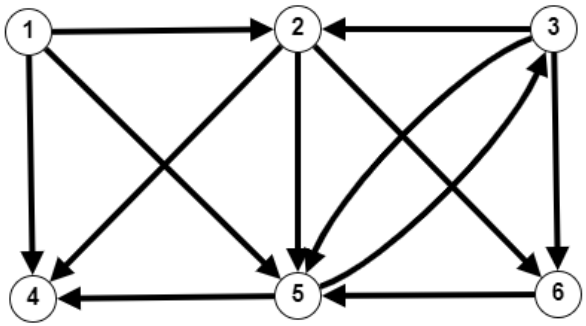
- For every vertex u we will have three labels:
  - $d(u)$ the length of the shortest $s \rightsquigarrow u$ path found
    $d(u) = \infty$ means no such path has been found so far
    $d(u) = 0$ is only true for $u = s$
  - $\pi(u)$ the parent vertex of u on the $s \rightsquigarrow u$ path found.
    $\pi(u) = \oslash$ means that we have not found such path or $u = s$
  - $color[u]$ can be omitted from the program (useful for illustration)
    white if it has not been found yet
    grey if it has been found but not yet processed
    black if it has already been processed
- In the first loop of the algorithm these labels are set:
  $\forall u \in G.V$: $d(u) = \infty$, $\pi(u) = \oslash$ and $color(u) = white$
  for s source: $d(s) = 0$, $\pi(s) = \oslash$, $color(s) = grey$

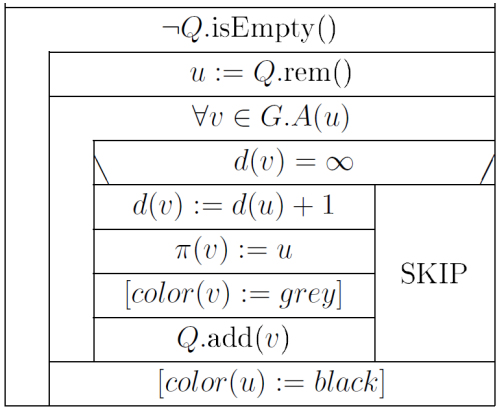
Breadth-first search



- s is added to queue Q
- In every iteration a vertex u is removed from Q
- and "expanded", i.e. all its neighbours are considered, and if they have not been found before ($d(v) = \infty$) then their d, π and *color* labels are set, and they are added to Q
- Expanded vertices become black.

Example for BFS



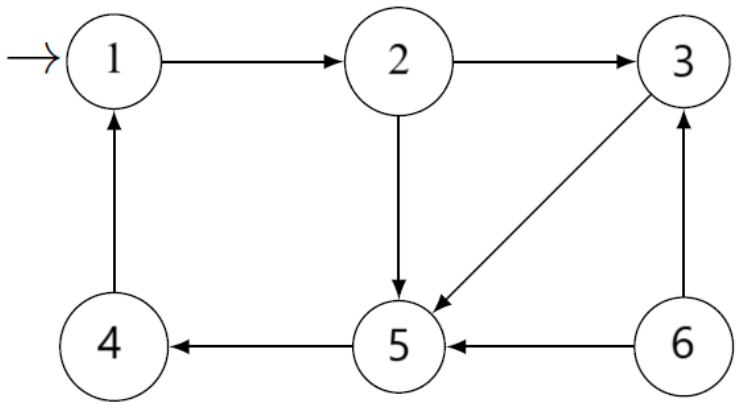


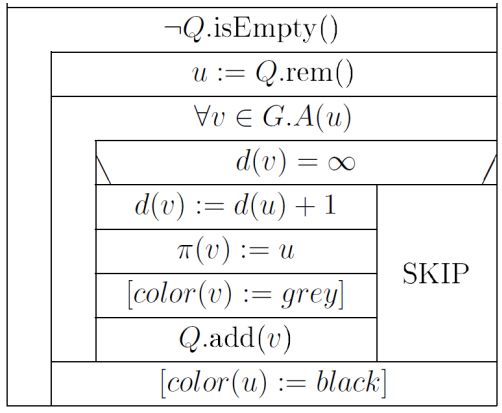
changes of d						ex-	Q :	changes of π					
1	2	3	4	5	6	panded	Queue	1	2	3	4	5	6
						vertex : d	$\langle \quad \rangle$						
						final d and π values							

Properties

- During the algorithm when a vertex v with value $d(v) = k$ is being expanded (or processed), then all the vertices in the queue Q have d value k or $k + 1$.
- When the last vertex with d value k is processed, then all the vertices in Q have d value $k + 1$.
- The set of vertices with d value k are the vertices on level k of the breadth-first tree.
- Edges $(\pi(u), U)$ form the **breadth-first tree** or **shortest paths tree.**

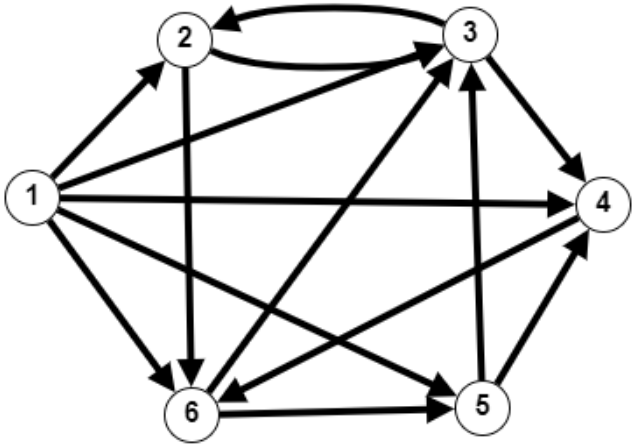
Example for BFS





changes of d						ex- panded vertex : d	Q : Queue	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
							$\langle \quad \rangle$						
						final d and π values							

Example for BFS

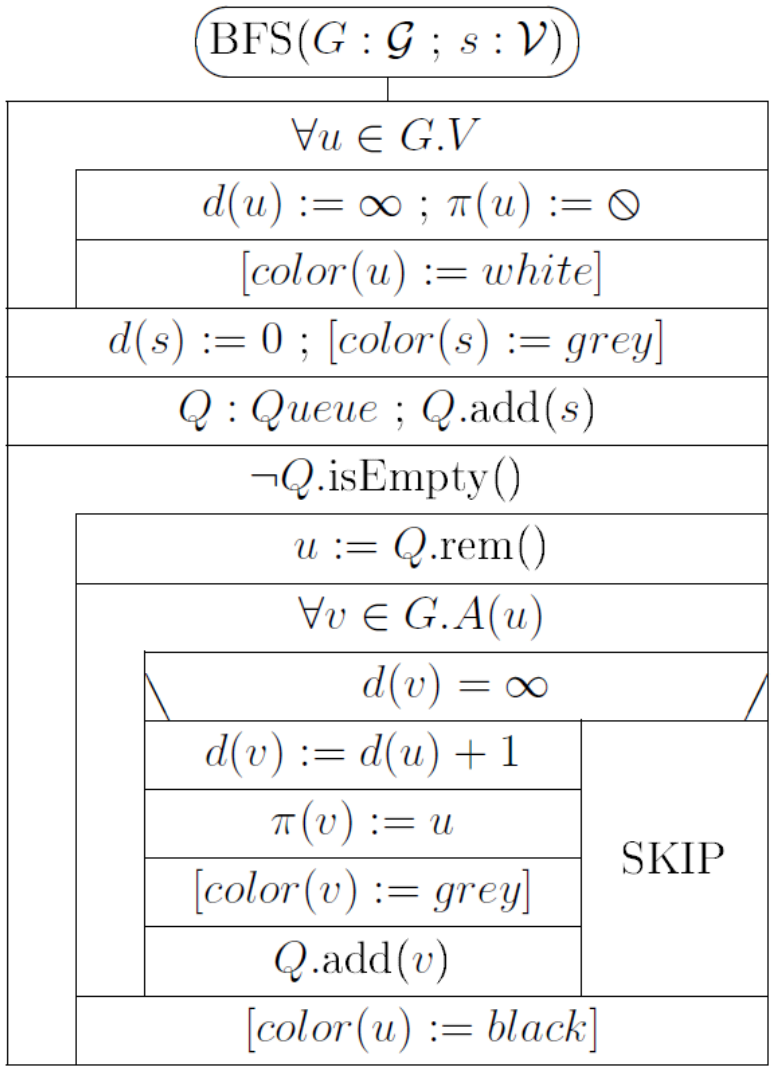


changes of d						ex- panded vertex : d	Q : Queue	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
							$\langle \quad \rangle$						
						final d and π values							

Optional assignment

Let us suppose that we have run precedure $BFS(G, s)$. Write an algorithm $printShortestPathTo(v)$ which prints the shortest path $s \rightsquigarrow v$ that has been calculated by BFS if v is available from vertex s, and prints *"v is not avalable from s"* otherwise. Both recursive and non recursive algorithms can be given to solve this problem.

Efficiency of BFS



- $n = |G.V|$ and $m = |G.E|$
- Initializing loop iterates n times.
- The main loop iterates as much as the number of vertives are vailable from s (which is min 1, max n).
- The inner loop iterates max m or $2m$ times and min is 0.
- Time complexity: $MT(n, m) \in \Theta(n + m)$, $mT(n, m) \in \Theta(n)$

Table of Contents

Transitive closure

For each pair of vertices (u, v) of a network/graph, we want to determine whether there is any path form u to v or not. We are interested neither in the path nor in its length. For this purpose we introduce the following notion.

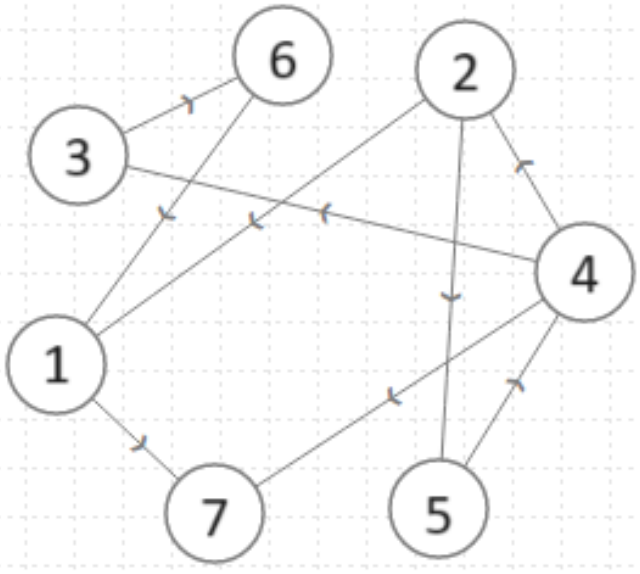
Definition: Given graph $G = (V, E)$ its **transitive closure** is relation $T \subseteq V \times V$ where

$(u, v) \in T \iff \exists$ path from vertex u to vertex v in graph G.

Provided that the vertices of a graph can be identified by indices $1, 2, \ldots, n$, we can represent the graph with adjacency matrix $A/1 : \mathbb{B}[n, n]$.

And we can represent its transitive closure with matrix $T/1 : \mathbb{B}[n, n]$, where

$T[i, j] = \mathit{true} \iff \exists$ path$i \rightsquigarrow j$ in the graph repr. with mtx A.



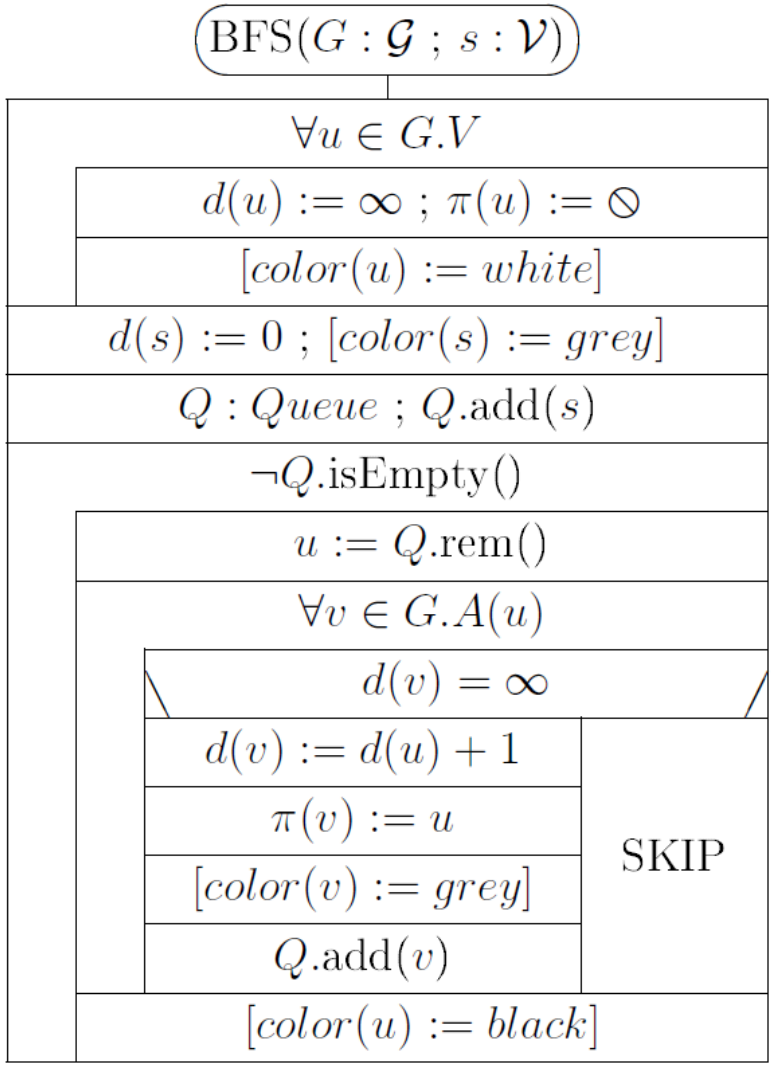
Directed Graph

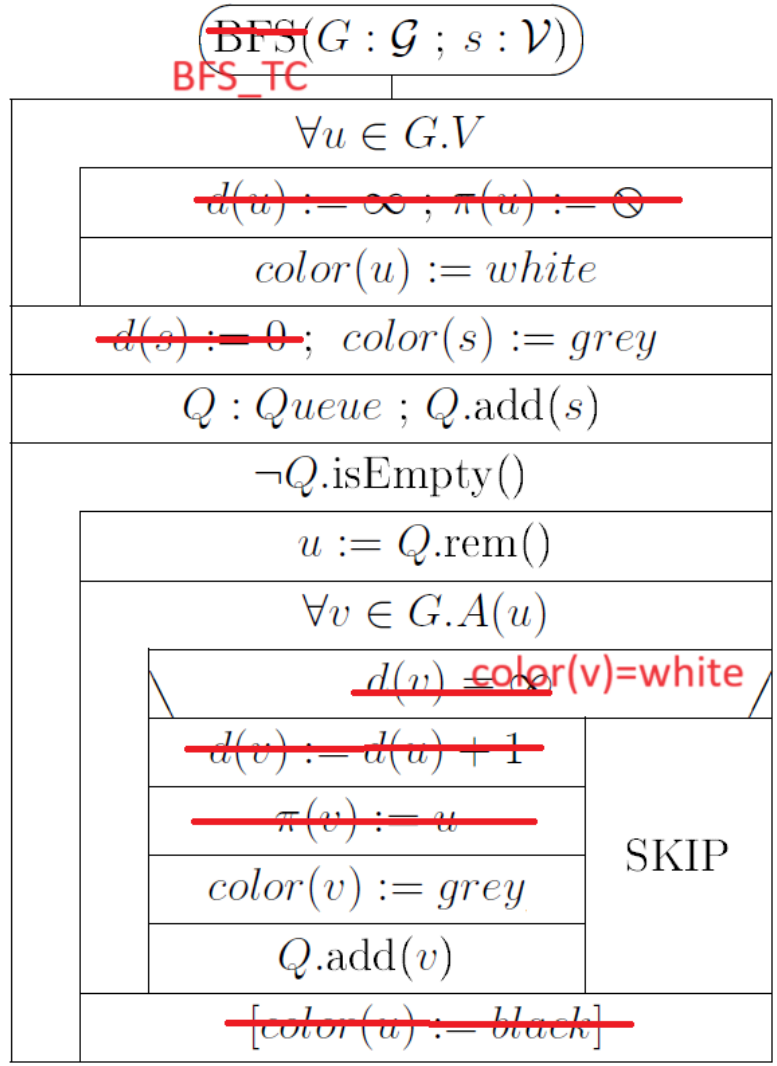
	1	2	3	4	5	6	7
1	1	0	0	0	0	0	1
2	1	1	1	1	1	1	1
3	1	0	1	0	0	1	1
4	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1
6	1	0	0	0	0	1	1
7	0	0	0	0	0	0	1

Transitive Closure

How can we calculate this matrix T?

First solution: perform a BFS from each vertex as source. We do not need to compute distances, nor parent vertices, but only reachability. In BFS a vertex is reached if it is not white.





After running algorithm BFS_TC with source vertex i we have

$$T[i,j] = \mathit{true} \iff \text{color(j)=grey}$$

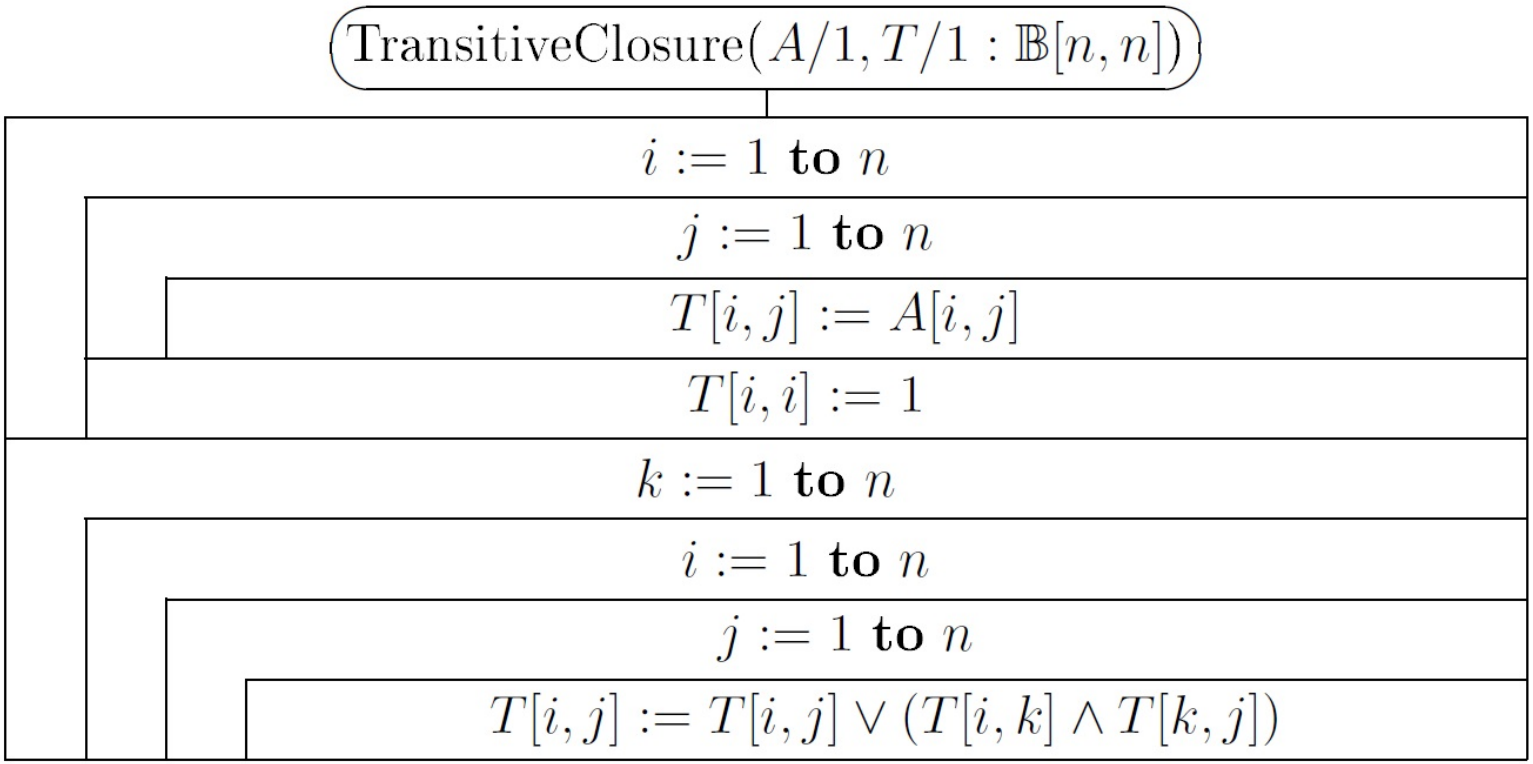
We obtain row i of matrix T.
If we run this algorithm for all vertices i, then we obtain T.

The time complexity will then be n times $\Theta(m+n)$, which is

$$\Theta(n(n+m))$$

For sparse graphs this is $\Theta(n^2)$, but for dense graphs it is $\Theta(n^3)$.

Another way of calculating transitive closure:

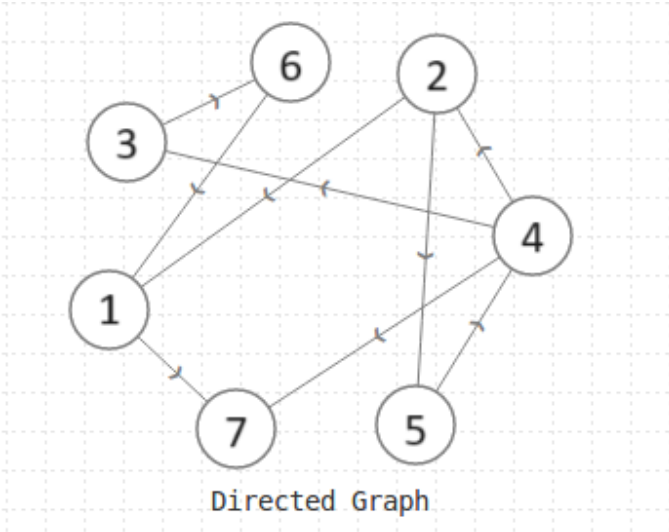


Clearly the time complexity for this algorithm is $\in \Theta(n^3)$.

Why is this algorithm good for computing Transitive Closure?

Notation. Let $i \overset{k}{\rightsquigarrow} j$ denote a path from vertex i to vertex j which uses only vertices $\leq k$ as inner vertices.
($i > k$ and/or $j > k$ is possible.)

So the path can be $< i, u_1, u_2 \ldots, u_n, j >$ where $u_1, u_2, \ldots, u_n \leq k$.



In this graph there is no $2 \overset{1}{\rightsquigarrow} 3$ path, nor $2 \overset{4}{\rightsquigarrow} 3$ or $2 \overset{0}{\rightsquigarrow} 3$ paths, but there is $2 \overset{5}{\rightsquigarrow} 3$ path. There is no $3 \overset{5}{\rightsquigarrow} 1$ path, but there is $3 \overset{6}{\rightsquigarrow} 1$ path.

Definition: $T^{(k)} = \mathit{true} \iff \exists\ i \overset{k}{\rightsquigarrow} j$ where $k \in 0...n$ and $i, j \in 1..n$

For every $k = 0, 1, ..., n$ we will compute matrix $T^{(k)}$, and clearly

$$T^{(n)} = T.$$

Recursive relation among the T matrices:

- $T_{ij}^{(0)} = A_{ij} \vee (i = j)$ for $i, j \in 1..n$
- $T_{ij}^{(k)} = T_{ij}^{(k-1)} \vee (T_{ik}^{(k-1)} \wedge T_{kj}^{(k-1)})$ for $k, i, j \in 1..n$

Use these properties to prove that the following algorithm produces the matrix $T = T^{(n)}$.

```
i := 1 to n
    j := 1 to n
        T^(0)[i, j] := A[i, j]
    T^(0)[i, i] := 1
k := 1 to n
    i := 1 to n
        j := 1 to n
            T^(k)[i, j] := T^(k-1)[i, j] ∨ (T^(k-1)[i, k] ∧ T^(k-1)[k, j])
```

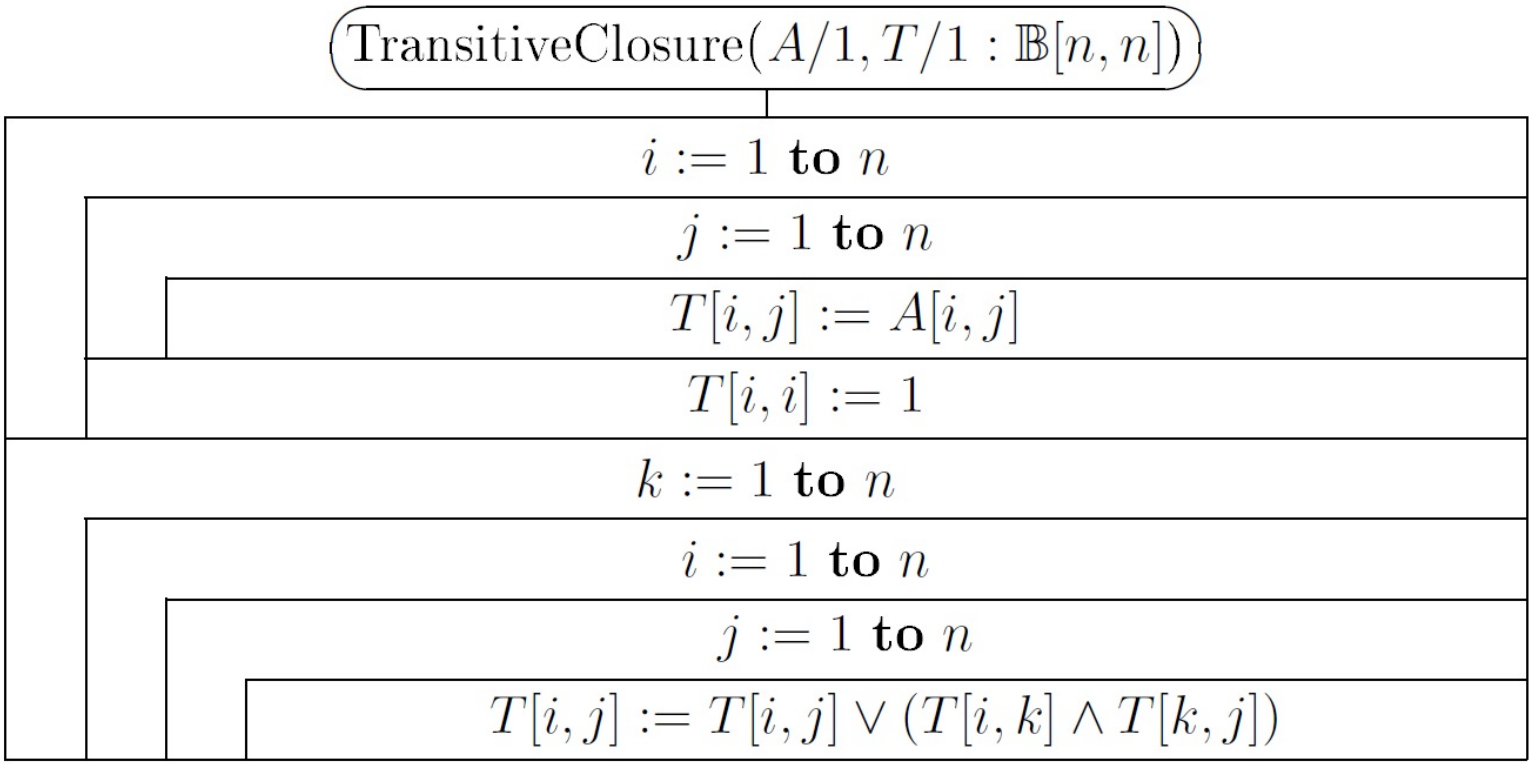
Do we really need all the $T^{(k)}$ matrices?
No, we don't.

Observation: From the recursive relations we have:

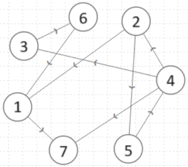
- $T_{ik}^{(k)} = T_{ik}^{(k-1)}$
- $T_{kj}^{(k)} = T_{kj}^{(k-1)}$

This means that column k and row k of matrix $T^{(k)}$ are the same as the appropriate column and row of matrix $T^{(k-1)}$.
From this it follows that a single matrix T is enough for the whole computation, because $T_{ij}^{(k)}$ depends only on $T_{ij}^{(k-1)}$, $T_{ik}^{(k-1)}$, $T_{kj}^{(k-1)}$.

So this algorithm will also calculate the transitive closure:



Clearly the time complexity for this algorithm is $\in \Theta(n^3)$.



Directed Graph

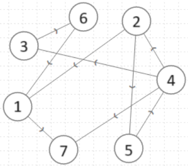
	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							



Directed Graph

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1	1	0	0	0	0	0	1
2	1	1	1	1	1	1	1
3	1	0	1	0	0	1	1
4	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1
6	1	0	0	0	0	1	1
7	0	0	0	0	0	0	1

Transitive Closure